

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄静怡 学号 2024K8009907042 专业 计算机科学与技术
实验项目编号 3 实验名称 定制功能型处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 状态机设计

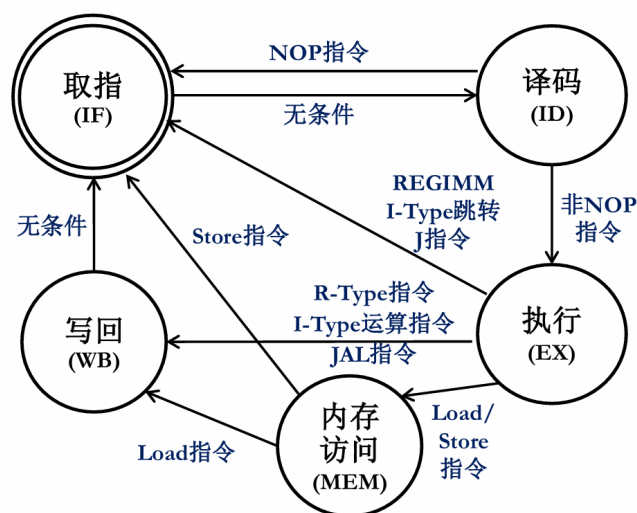


图 1: 状态机转移图

根据讲义要求, 状态机使用“三段式”进行描述

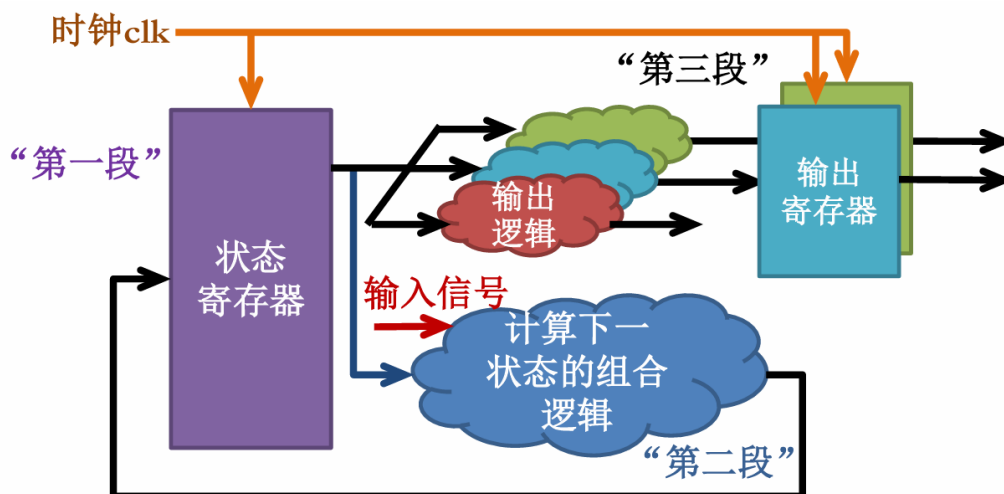


图 2: 状态机“三段式”

“第一段”用 always 时序逻辑,描述状态寄存器的同步状态跳转

“第二段”用 always 组合逻辑,根据状态机当前状态和输入信号,描述下一状态的计算逻辑

“第三段”用 always 时序逻辑或 assign 组合逻辑,根据状态机当前状态,描述不同输出寄存器的同步变化

2 puts 函数与握手

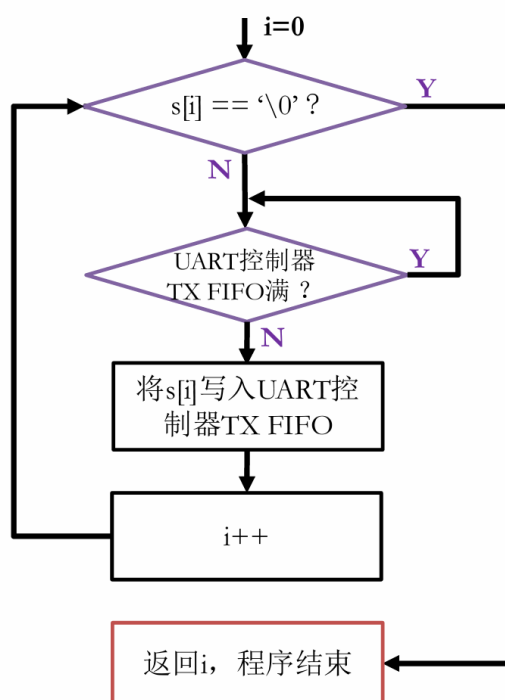


图 3: puts 函数流程图

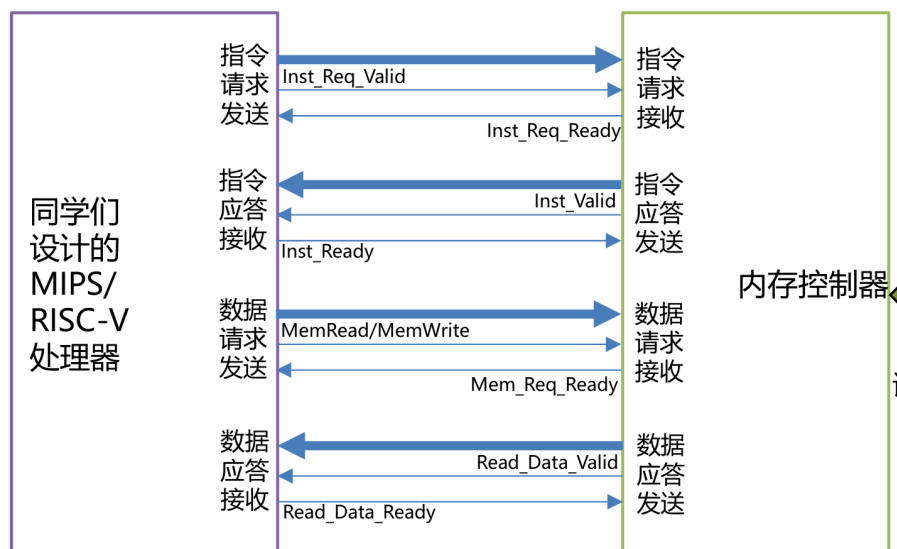


图 4: 握手示意图

二、 开发成果(对代码开发流程的总结。需附上包含注释的 VerilogHDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 cpu 文件设计

1.1 状态机设计

状态机的状态定义

```
localparam
INIT = 9'b0000000001,
IF  = 9'b0000000010,
IW  = 9'b0000000100,
ID  = 9'b0000001000,
EX  = 9'b0000010000,
ST  = 9'b0000100000,
LD  = 9'b0001000000,
RDW = 9'b0100000000,
WB  = 9'b1000000000;
```

状态机第一段

```
//定义状态转移
always @(posedge clk) begin
    if(rst) begin
        current_state <= INIT;
    end
    else begin
        current_state <= next_state;
    end
end
```

在每个时钟信号上升沿到来的时候, 将 `current_state` 的值赋值为 `next_state`, 这样就实现了状态机的转移.
状态机第二段

//定义`next_state`, 利用连续赋值`always @(*)`, 此处主要的问题是比较复杂的`EX`状态

```
always @(*) begin
    case(current_state)
        INIT: next_state = IF;
        IF: begin
            if(Inst_Req_Ready) begin
                next_state = IW;
            end
            else begin
                next_state = IF;
            end
        end
        IW: begin
            if(Inst_Valid) begin
                next_state = ID;
            end
            else begin
                next_state = IW;
            end
        end
        ID: begin
            if(Instruction_current[31:0] == 32'b0) begin
                next_state = IF;
            end
            else begin
                next_state = EX;
            end
        end
        EX: begin
            if(opcode==`REGIMM || opcode[5:2]==4'b0001 || opcode==`J) begin
                //表示直接返回取指阶段的三种指令类型, REGIMM, I跳转和J
                next_state = IF;
            end
            else if (opcode[5:3]==3'b101) begin
                next_state = ST;
            end
            else if (opcode[5:3]==3'b100) begin
                next_state = LD;
            end
            else begin
                next_state = WB;
            end
        end
        ST: begin
            if(Mem_Req_Ready) begin
                next_state = IF;
            end
        end
    endcase
end
```

```

        else begin
            next_state = ST;
        end
    end
end
LD: begin
    if(Mem_Req_Ready) begin
        next_state = RDW;
    end
    else begin
        next_state = LD;
    end
end
RDW: begin
    if(Read_data_Valid) begin
        next_state = WB;
    end
    else begin
        next_state = RDW;
    end
end
WB: begin
    next_state = IF;
end
default : begin
    next_state = INIT;
end
endcase
end

```

状态机第二段描述次态的计算组合逻辑,状态机第二段只对 next_state 赋值。

状态机第三段

```

assign Inst_Req_Valid = (current_state == IF);
assign Inst_Ready     = (current_state == INIT || current_state == IW);
assign MemRead        = (current_state == LD);
assign MemWrite       = (current_state == ST);
assign Read_data_Ready = (current_state == RDW || current_state == INIT);

```

根据当前状态来对这些控制信号进行赋值,这里利用组合逻辑来避免时序出现错误,要注意 INIT 状态的时候拉高 Inst_Ready 和 Read_data_Ready,来避免复位释放后造成的影响。

1.2 指令寄存器的设计

```

//取指阶段的握手机制,只有在握手成功的时候,才更改当前的指令
reg [31:0] Instruction_current;
always @(posedge clk) begin
    if(rst) begin
        Instruction_current <= 32'b0;
    end
end

```

```

else if(Inst_Ready && Inst_Valid) begin
    Instruction_current <= Instruction;
end
else begin
    Instruction_current <= Instruction_current;
end
end
end

```

我们只有在 IW 阶段 Inst_Valid 信号与 Inst_Ready 信号握手成功时, 才会将当前的指令 Instruction_current 赋值为内存读出的数据 Instruction, 其他时候保持不变, 来避免之后的 ID 以及 EX 等部分的组合逻辑信号出现错误。

1.3 PC 部分

```

always @(posedge clk) begin
    if (rst) begin
        PC_reg <= 32'b0;
    end
    else if(Instruction_current==32'b0 && current_state==ID) begin //表示NOP指令,
        在译码后直接回到取指阶段
        PC_reg <= PC_reg + 4;
    end
    else if(current_state==EX) begin //对于其他类型的指令, 根据指令类型来判断跳转
        PC_reg <= (Jump?    Jump_addr:
            Branch_f? PC_reg + Branch_addr + 4:
                PC_reg + 4);
    end
    else begin
        PC_reg <= PC_reg; //对于其他状态, PC不改变
    end
end
end

```

要注意的是, 我们在对指令 JAL 以及 JALR 的阶段写回时, 用的是未更新的 PC, 所以需要有一个寄存器用来存储改变前的 PC

```

reg [31:0] old_PC;

always @(posedge clk) begin
    if(rst) begin
        old_PC <= 32'b0;
    end
    else if (current_state==IF) begin
        old_PC <= PC_reg;
    end
    else begin
        old_PC <= old_PC;
    end
end
end

```

1.4 译码阶段和执行阶段

这一部分的修改比较简单, 只需要改成时序逻辑和加上复位, 其他的和单周期 cpu 完全一样

1.5 写回阶段

```
always @(posedge clk) begin
    if(rst) begin
        Read_data_current <= 32'b0;
    end
    else if(Read_data_Ready && Read_data_Valid) begin
        Read_data_current <= Read_data;
    end
    else begin
        Read_data_current <= Read_data_current;
    end
end
```

1.6 指令周期计数器

```
reg [31:0] cycle_cnt;

always @(posedge clk) begin
    if(rst) begin
        cycle_cnt <= 32'd0;
    end
    else begin
        cycle_cnt <= cycle_cnt + 32'd1;
    end
end

assign cpu_perf_cnt_0 = cycle_cnt;

reg [31:0] ins_cnt;

always @(posedge clk) begin
    if(rst) begin
        ins_cnt <= 32'd0;
    end
    else if (Inst_Ready && Inst_Valid )begin
        ins_cnt <= ins_cnt + 32'd1;
    end
    else begin
        ins_cnt <= ins_cnt;
    end
end

assign cpu_perf_cnt_1 = ins_cnt;
```

```

reg [31:0] mem_cnt;

always @(posedge clk) begin
    if(rst) begin
        mem_cnt <= 32'd0;
    end
    else if (Read_data_Ready && Read_data_Valid)begin
        mem_cnt <= mem_cnt + 32'd1;
    end
    else begin
        mem_cnt <= mem_cnt;
    end
end

assign cpu_perf_cnt_2 = mem_cnt;

```

2 puts 函数

```

int
puts(const char *s)
{
    int count = 0;
    while (s[count] != '\0')
    {
        while (uart[UART_STATUS >> 2] & UART_TX_FIFO_FULL);
        uart[UART_TX_FIFO >> 2] = s[count];
        count++;
    }
    return count;
}

```

实现向 UART 控制器传送字符串的程序功能,把一段字符串,通过串口一个字节一个字节发出去,并且返回发送了多少个字符。

3 性能计数器

```

//perf_cnt函数
unsigned long _uptime() {
    return (unsigned long)PERF_CNT_BASE[0];
}

unsigned long _read_mem_cycle() {
    return (unsigned long)PERF_CNT_BASE[1];
}

void bench_prepare(Result *res) {
    start_cycle = _uptime();
}

```



```

start_mem_cycle = _read_mem_cycle();
res->msec = 0;
res->mem_cycle = 0;
}

void bench_done(Result *res) {
    res->msec = _uptime() - start_cycle;
    res->mem_cycle = _read_mem_cycle() - start_mem_cycle;
}

```

uptime 读硬件计时器, 返回系统总运行时间

read_mem_cycle 读内存访问周期计数器, 返回执行内存读写消耗的周期数

bench_prepare 测试开始前调用, 记录开始时间、开始内存周期, 把结果清零

bench_done 测试结束后调用, 计算: 总耗时 = 结束时间 - 开始时间, 总内存周期 = 结束内存周期 - 开始内存周期

三、 学习总结(总结设计、开发与调试过程中遇到的问题, 及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

1. 在本次实验中的时序也是很容易出现问题的, 因此要严格按照课上讲义的状态机来编写时序, 我在本次实验中添加了译码阶段和执行阶段的寄存器, 用于尽可能避免时序提前

2. 在本次实验中出现的另一大问题是 fpga 的编码, 我一开始对 next_state 赋值是用的非阻塞赋值, 这样是过不了 fpga 的测试的, 因为实际上 always@(*) 所描述的是组合逻辑, 所以不能用非阻塞赋值, 应该用阻塞赋值来实现连续赋值

3. 在本次实验中出现的 always 块中运用了很多 case 和 if-else 语句, 我们在编写的时候要注意 if-else 枚举全部情况, case 要写 default

4. 实验中也犯了不少低级语法错误, 教师的报错系统帮助我快速定位解决问题。

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	时序	译码执行	时序逻辑有问题	时序提前	添加寄存器
2	fpga 编码	状态机	非阻塞赋值有问题	组合逻辑不可使用非阻塞	用阻塞赋值
3	代码编写	状态机	情况不完全	情况不完全	枚举全部情况
4	语法错误	所有版块	对代码编写还不算熟练	实战经验较少	加强练习

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

在本实验中, 教师团队引入了报错分析, 帮助很多, 更快地帮助我定位并解决了代码问题。

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

• volatile 关键字的作用是什么? 如果去掉会出现什么后果?。

volatile 这个单词的本义是指不稳定的, 多变的, 这个在 C 程序的关键字我认为其作用是控制编译器, 让编译器在编译的时候不要对这个语句在编译中进行优化, 让 C 程序编译成机器语言的时候保持其 C 程序本身的语

类别	名称	体会	后续计划
对 cpu 的理解	理解多周期 cpu	更加熟悉 cpu 运行原理,即几个阶段的整合	多复习讲义
代码编写	编写 verilog 并实现 always 逻辑	代码编写更加熟练	编写代码
debug 能力	寻找错误	寻找代码错误原因更加熟练	辅助工具实现 debug

表 2: 收获总结

句 (比如不将两个语句合成一个, 不将多条访存语句合并成一条), 这样可以帮助我们可以从波形中准确的知道内存的操作, 更好观察.

五、 实验时长

在课后,你花费了大约 12 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	3	认真学习课程内容,理解所需实现的板块
开发	4	完成实验代码的编写,包括多周期 cpu 文件,计数器等等
调试	5	进行测试样例的测试,不断修正错误

表 3: 实验时长